

// ===== ./docs/huawei_login_design.md =====

大蓝书 · 华为 Account Kit 一键登录 · 系统设计与任务分解

> 角色：交付架构师 高见远 (Bob)

> 范围：P0 全量 + P1 ①④ (首次昵称/头像初始化、深色适配)；P1 ②③ (失败降级、unionID 冲突) 已登录/埋点) 不排期

> 技术栈：后端 Node.js + Express + TypeScript + Prisma + MySQL (端口 3000)；前端 HarmonyOS EXT 原生 (ArkTS + ArkUI, Stage 模型 **API 24**，ArkTS V1 严格模式)

> 配套图：`docs/huawei\_login\_sequence.mermaid` (调用时序)、`docs/huawei\_login\_class.mermaid` (类/结构)

0. 已核实现事实与对 PRD 假设的必要修正 (重要, 请先读)

我直接阅读了现有代码, 发现 PRD/前置说明中有多处描述与**实际代码**不符。以下为权威事实, 后续设计以

#	PRD/前置假设	实际代码 (已核实)	对本次设计的影响
F1	后端 `auth.ts` 是「手机号/账号密码登录」	实际收 `{openId, nickname}`; 是**鸿蒙 openId 授权登录** (当前为 dev 桩 `科技老张`); 仓库内**无任何手机号/密码端点**	登录 + 新增华为 unionID 登录**
F2	前端「复用 AuthManager 持久化」	前端 `utils/auth.ets` 是**模块级函数** (`setSession`, `getToken`, `getCurrentUser`, `clearSession`, `ensureLogin`), **不是类**	华为登录成功后调用 `setSession(token, user)` 即可复用同一登录态
F3	假设存在 `LoginPage.ets`	仅有 `Index` / `DetailPage` / `SearchResultPage`	登录是各页 `ensureLogin()` dev 桩**自动**完成, **无真实登录 UI**
F4	`main_pages.json` 在 `ets/` 下	实际路径 `entry/src/main/resources/base/profile/main_pages.json`	注册 `LoginPage` 改此文件
F5	后端用 axios / fetch (待定)	原生 `fetch` 全局可用	华为 OAuth 调用用**原生 `fetch`**
F6	`User` 加 `unionID(unique)`	当前为 `String @unique` (NOT NULL)	华为用户无 openId 须把 `openId` 改为 `String? @unique` (可空唯一), 并新增 `unionID`
F7	华为端点响应结构待定	现有登录响应为 `{token, user}`	前端零改解析

1. 实现方案 + 框架选型

结论：完全沿用现有栈, 仅做最小增量改动, 不引入新框架/新依赖 (前端零新增、后端零新增)。

1.1 前端: 系统组件 `LoginWithHuaweiIDButton`

- 来自 `@kit.AccountKit`, 属 HarmonyOS SDK **内置模块**, **无需 ohpm 安装**。
- 它是 ArkUI 系统级按钮: 用户点击 → 华为系统授权 (指纹/密码) → 回调返回**一次性 `authorization\_code`**
- 在 `LoginPage.ets` 的 `build()` 中嵌入该按钮, 并配 `onLoginSuccess` / `onLoginFailure` 回调
- **深色模式 (P1-④)**: `LoginWithHuaweiIDButton` 支持 `mode` 参数 (亮/暗/跟随系统), 将 `pp` 当前色系即可; 系统组件默认也能跟随系统深色, 本任务仅做显式绑定确认。

1.2 后端: 新增 `huaweiAuth` 服务 (原生 `fetch`)

- 选 **原生 `fetch`** (依据 F5: 无 axios、Node v22 全局可用), 不新增依赖, 保持依赖面最小。
- 封装两次华为调用:
 1. **换 token**: `POST https://oauth-login.cloud.huawei.com/oauth2/v3/token`
 - `grant\_type=authorization\_code`、`client\_id`、`client\_secret` (从 `process.env.HUAWEI\_CLIENT\_SECRET` 读取)、`code`;
 - `redirect\_uri` 可选 (AGC 注册回填, 通常留空)。
 2. **取 unionID + 资料**: `GET https://account.cloud.huawei.com/rest.php?nsp\_svc=G...`

- 请求头 `Authorization: Bearer <access\_token>`, 返回 `{ unionID, nickName, avatarUri, ... }`。
- `client\_secret` **只在后端 `.env` 读取, 绝不下发前端**。

1.3 并存策略 (核心)

- 华为 unionID 登录与现有 openId 登录 **共用同一套 JWT token 体系与 `setSession` 持久化**。
- 后端新增 `loginWithHuawei(unionID, nickname?, avatar?)`: `findOrCreate` (按 `unionID` 唯一查; 命中返回原账号--「首次关联为准」; 未命中则创建)。
- 归档原则: 沿用后端既有「route → service → prisma」三层; 前端沿用「模块函数鉴权工具 + `api`」。

----- 第 1 页 -----

2. 文件列表及相对路径 (标注 新增 / 修改)

文件	操作	说明
backend/prisma/schema.prisma	改	`User.openId` → `String? @unique`; 新增 `unionID String? @unique`
backend/src/config/env.ts	改	`env` 新增 `huawei: { clientId, clientSecret, redirectUri }`
backend/src/utils/response.ts	改	`CODE` 新增 `HUAWEI_AUTH_FAILED: 480`
backend/.env	改	新增 `HUAWEI_CLIENT_ID=` / `HUAWEI_CLIENT_SECRET=` / `HUAWEI_REDIRECT_URI=` (不提交, 已 gitignore)
backend/.env.example	改	同步新增三项占位 (模板, 可提交)
backend/src/services/huaweiAuth.ts	**新**	封装 OAuth 换 token + getInfo 取 unionID
backend/src/services/authService.ts	改	新增 `loginWithHuawei(unionID, nickname?, avatar?)`
backend/src/routes/auth.ts	改	新增 `POST /huawei/exchange` (`/v1/auth` 与 `/v1/users` 已双挂)
entry/src/main/ets/services/api.ets	改	新增 `HuaweiExchangeBody` 类型 + `exchangeHuaweiCode`, 复用 `LoginResult`
entry/src/main/resources/base/profile/main_pages.json	改	`src` 数组新增 `pages/LoginPage`
entry/src/main/ets/pages/Index.ets	改	`aboutToAppear`: 无 session 时 `router.pushUrl({url: 'pages/LoginPage'})`
entry/src/main/ets/pages/LoginPage.ets	**新**	登录页: 嵌入 `LoginWithHuaweiIDToken` + 原方式入口
entry/src/main/ets/components/HuaweiLoginButton.ets	**新**	封装系统按钮 + 回调 `LoginPage` 复杂度, 便于深色/降级复用
entry/src/main/ets/utils/auth.ets	改	增加华为路径辅助 (如 `loginByHuaweiResult` 兼容 `ensureLogin` 兼容), 不改现有签名语义

> 注: 以上相对路径基于 `HarmonyProject1/` 根目录。`entry/src/main/module.json5` **默认不识别 Client ID 与 App ID 相同时无需配 metadata); 若 AGC 中两者不同, 再补 `metadata.clientID`

3. 数据结构和接口

3.1 `User` 模型变更 (`prisma/schema.prisma`)

```
prisma
model User {
  id          Int          @id @default(autoincrement())
  openId      String?      @unique @db.VarChar(64) // 鸿蒙 openId (华为登录用户可为空)
  unionID     String?      @unique @db.VarChar(64) // 华为 unionID (新增, 唯一)
  nickname    String        @db.VarChar(32)
  avatar      String?      @db.VarChar(255)
  gender      Int?          @default(1)
  followTags  Json?
```

```

    status      Int      @default(1)
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt
    // 关联 (posts/comments/...) 保持不变
}

```

> MySQL 唯一索引允许多个 NULL，故 `openId`/`unionID` 可空 + 唯一成立；现有 openId 用户不受影响

3.2 接口结构

****请求**** `POST /v1/auth/huawei/exchange`

```

```json
{ "code": "string" } // 华为一次性 authorizationCode

```

----- 第 2 页 -----

**\*\*响应\*\*** (复用现有 `LoginResult`)

```

```json
{ "code": 0, "data": { "token": "string", "user": { "id": 1, "nickname": "华为用户xx", "avatar": "https://..." } }, "message": "success" }

```

失败时 `code: 480` (见 7.1), `httpStatus: 200` (保证前端 `request()` 能抛出友好文案, 见 3.4)

3.3 后端方法签名 (关键签名, 非完整实现)

```

```ts
// backend/src/services/huaweiAuth.ts
export async function exchangeCodeForToken(code: string): Promise<string>;
// → access_token
export async function fetchHuaweiUserProfile(accessToken: string): Promise<{ unionID: string; nickName?: string; avatarUri?: string }>;

```

```

// backend/src/services/authService.ts (新增, 复用 jwt 签发)
export async function loginWithHuawei(unionID: string, nickname?: string, avatar?: string): Promise<{ token: string; user: User }>;

```

### ### 3.4 路由 (关键片段)

```

```ts
// backend/src/routes/auth.ts
router.post('/huawei/exchange', async (req, res) => {
  const { code } = req.body ?? {};
  if (!code) return fail(res, CODE.BAD_REQUEST, '缺少 authorizationCode');
  try {
    const token = await exchangeCodeForToken(code);
    const profile = await fetchHuaweiUserProfile(token);
    const result = await loginWithHuawei(profile.unionID, profile.nickName, profile.avatarUri);
    return ok(res, result); // { code:0, data:{token, user:User}}
  } catch {
    // 返回 200 + 业务码 480, 前端 request() 抛 message → toast (P1-② 降级不阻塞)
    return fail(res, CODE.HUAWEI_AUTH_FAILED, '华为登录失败, 请稍后重试或切换其他登录方式');
  }
});

```

3.5 前端 API 封装 (`api.ets` 新增)

```

```ts
export interface HuaweiExchangeBody { code: string; }
export function exchangeHuaweiCode(body: HuaweiExchangeBody): Promise<LoginResult> {
 return api.post<LoginResult>('/v1/auth/huawei/exchange', body);
}

```

...

### ### 3.6 前端登录页（`LoginPage.ets` 关键片段，严格模式友好写法）

```
import { LoginWithHuaweiIDButton } from '@kit.AccountKit';
import { exchangeHuaweiCode } from '../services/api';
import { setSession, getCurrentUser } from '../utils/auth';

struct LoginPage {
 ----- 第 3 页 -----
 aboutToAppear(): void { /* initAuthStorage 已在 Index 调；此处无需重复 */ }
 onHuaweiSuccess(resp: AuthorizationWithHuaweiIDResponse): void {
 const code: string = resp.authorizationCode;
 exchangeHuaweiCode({ code }).then((r: LoginResult) => {
 setSession(r.token, r.user); // 复用现有登录态
 router.back(); // 回到 Index (已登录)
 }).catch((e: Error) => { /* toast: e.message, 不阻塞 */ });
 }
 onHuaweiFailure(error: Error): void { /* toast 降级，保留“原方式”入口 */ }
 build(): void {
 Column() {
 LoginWithHuaweiIDButton({
 params: { /* scopeList 可选；默认即可换 code */ },
 onLoginSuccess: (resp: AuthorizationWithHuaweiIDResponse): void => { thi
s.onHuaweiSuccess(resp); },
 onLoginFailure: (error: Error): void => { this.onHuaweiFailure(error); }
 }).mode(/* 绑定 App 色系: LoginWithHuaweiIDButtonMode */)
 Button('其他方式登录').onClick(): void => { /* 调 ensureLogin() 走现有 openId 路
*/ }
 }
 }
}
```

> 严格模式提示：回调沿用现有 `api.ets` 的箭头函数风格（现有 `api.get = <T>(p) => ...` 即此风格）  
`onLoginSuccess`/`onLoginFailure` 先转调 struct 方法，逻辑不写在匿名函数体内，降低审查风险。按钮 `param` 属性名以 API 24 官方文档为准\*\*（Account Kit 迭代较快）。

---

## ## 4. 程序调用流程

完整时序见 `docs/huawei\_login\_sequence.mermaid`。要点：

1. 用户进入 `LoginPage`（由 `Index.aboutToAppear` 判断无 session 后 `pushUrl` 而来）。
2. 点击 `LoginWithHuaweiIDButton` → 华为系统授权 → `onLoginSuccess(authorizationCode)`
3. 前端 `exchangeHuaweiCode({code})` → `POST /v1/auth/huawei/exchange`。
4. 后端 `huaweiAuth`：用 `code + client\_secret` 换 `access\_token` → 用 token 调 `getInfo` 取 `unionID/nickName/avatarUri`。
5. `authService.loginWithHuawei` 按 `unionID` \*\*findOrCreate\*\* → 签发 JWT → 返回 `{token, user}`。
6. 前端 `setSession(token, user)` 持久化 → `router.back()` 回 `Index`（已登录态）。
7. 失败：`onLoginFailure` 或后端 480 → 前端 toast 提示，\*\*保留「原方式」入口不阻塞\*\*（P1-②）。

类/结构关系见 `docs/huawei\_login\_class.mermaid`（User 模型、DTO、HuaweiAuthService、AuthService、AuthRoute、前端 ApiClient/AuthUtils/LoginPage/HuaweiLoginButton 及华为外部系统）。

---

## ## 5. 任务列表（有序、含依赖，按实现顺序）

> 规则遵循：≤5 个任务、每个任务 ≥3 个相关文件、按模块/层次分组、T01 为基础设施；依赖链仅两级。

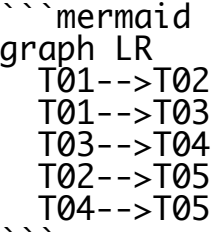
任务ID	任务名	归属	产物文件 (新增/修改)	依赖	优先级
****T01**	契约与配置基座 (后端)	后端	`prisma/schema.prisma` (改)、`src/config/env.` `src/utls/response.ts` (改)、`.env` (改)、`.env.example` (改); 执行 `prisma generate` + `prisma db push`	无	P0
****T02**	后端华为鉴权服务与路由	后端	`src/services/huaweiAuth.ts` (新)、`src/servi` `hService.ts` (改)、`src/routes/auth.ts` (改)	T01	P0
****T03**	前端 API 封装与页面路由	前端	`services/api.ets` (改)、`resources/base/pr` `main_pages.json` (改)、`pages/Index.ets` (改)	T01	P0
****T04**	前端登录页与华为按钮组件	前端	`pages/LoginPage.ets` (新)、`components/Hua` `Button.ets` (新)、`utls/auth.ets` (改)	T03	P0/P1

----- 第 4 页 -----

****T05**	联调·降级/深色/冲突验证	前后端	`pages/LoginPage.ets` (改)、`components/Hu` `inButton.ets` (改)、`utls/auth.ets` (改)	T02, T04	P1
-----------	---------------	-----	-------------------------------------------------------------------------------------	----------	----

- ##说明##
- \*\*\*\*T01 (先) \*\*：所有后续任务依赖此契约 (unionID 字段、env、错误码)。必须 `prisma db push` 否则 T02 运行即报错。
  - \*\*\*\*T02 (后端 service+route) \*\*：独立可并行于前端；实现 `findOrCreate` 即落实「首次关联为准」。
  - \*\*\*\*T03 (前端 api+路由) \*\*：仅加封装与页面注册，不影响现有页面。
  - \*\*\*\*T04 (前端登录页) \*\*：消费 T03 的 `exchangeHuaweiCode`；建议抽出 `HuaweiLoginButton.e` 系统按钮与回调，降低 `LoginPage` 复杂度。
  - \*\*\*\*T05 (联调/质量) \*\*：失败降级 toast (不阻塞原登录)、深色 `mode` 绑定、unionID 冲突策略真机 count Kit)。

### 5.1 任务依赖图



## 6. 依赖包列表

包	是否新增	说明
前端 `@kit.AccountKit`	****否**	(SDK 内置)   系统组件 `LoginWithHuaweiIDButton`，无需安装
后端 `axios`	****否**	依据 F5 改用原生 `fetch` (Node v22 全局可用)，不新增依赖
后端 `@prisma/client` / `prisma`	否	已有; T01 执行 `generate` + `db push`

- 需变更的环境变量 (不新增 npm 包)：
- `backend/.env`：新增 `HUAWEI\_CLIENT\_ID`、`HUAWEI\_CLIENT\_SECRET`、`HUAWEI\_REDIRECT\_URI` (可选)。`.env` 已被 `.gitignore` 忽略，不会提交。
  - `backend/.env.example`：同步新增三项占位，供团队成员复制。

## 7. 共享知识 (跨文件约定)

1. \*\*\*\*错误码约定\*\*：`CODE.HUAWEI\_AUTH\_FAILED = 480` (在 `response.ts` 新增)。所有华为登录 + 友好 `message`；HTTP 状态用 `200` (让前端 `request()` 抛出 `message` 而非底层 `HTTP 5xx`)
2. \*\*\*\*Token 字段命名\*\*：严格复用现有 `{ token, user }` (`LoginResult`)，`user` 含 `id/nickname/avatar`。\*\*\*\*禁止\*\*新增字段名 (如 `accessToken` / `huaweiToken`) 以免前端解析分歧。
3. \*\*\*\*登录态工具调用\*\*：用 `utls/auth.ets` 的 `setSession(token, user)` 持久化；持久化键固

oken / authUserId / authNickname / authAvatar` (PersistentStorage) 。\*\*不要\*\*新增登录态

4. \*\*HTTP 请求体\*\*：前端用 `@kit.NetworkKit` 的 `http`，body 走 `extraData` (严格模式铁律

i.ets` 的 `request()` 封装，\*\*不要\*\*手写 `fetch`/裸 `http`。

5. \*\*跨页参数\*\*：用 `AppStorage` (如 `setOrCreate('authToken', ...)`), \*\*禁止\*\* `route

Params()` (API 24 失效)。`LoginPage` → `Index` 回跳用 `router.back()`，无需传参。

6. \*\*密钥隔离\*\*：`HUAWEI\_CLIENT\_SECRET` 仅后端 `.env` / `.env.ts` 读取，\*\*绝不下发前端\*\*；

`authorizationCode`。

7. \*\*可空唯一键\*\*：`openId` 与 `unionID` 均为 `String? @unique` (见 F6)；`loginWithHua

号时 `openId` 留空、`unionID` 必填。

8. \*\*严格模式回调风格\*\*：`onLoginSuccess`/`onLoginFailure` 采用「箭头转调 struct 方法」写

ts` 现有箭头风格)，逻辑不放匿名函数体内。

9. \*\*深色模式\*\*：`LoginWithHuaweiIDButton.mode` 绑定 App 当前色系；系统组件默认跟随系统深色

P1-④)。

10. \*\*路由前缀\*\*：`authRouter` 同时挂 `/v1/auth` 与 `/v1/users`，新增 `/huawei/exchange

可访问，前端用 `/v1/auth/huawei/exchange`。

---

----- 第 5 页 -----

## 8. 待明确事项 (需用户 / 产品经理决策)

- \*\*unionID 冲突策略 (关键)\*\***：当前方案「首次关联为准」--`unionID` 已存在则直接返回原账号 (不  
」。需 PM 确认 UX：
  - (a) 静默用原账号登录 (当前默认, 最简单)；
  - (b) 弹窗提示「该华为账号已绑定其他大蓝书账号」, 要求换绑/解绑 (P2 再做换绑能力)。
- \*\*「原方式」UI 范围\*\***：实际代码中并无手机号/密码 UI (F1)。P0 的「两种方式并存」实现为「华为  
」的『其他方式登录』入口」即可满足 US-5；是否要真正做手机号/账号密码表单，\*\*不在本期 P0\*\*，需 PM
- \*\*登录页入口交互\*\***：当前 `Index` 是 `@Entry` 启动页。建议 `Index.aboutToAppear` 判断无  
`router.pushUrl('pages/LoginPage')` (Index 保留在栈底, 登录后 `router.back()` 回 Index)
- 人/PM 确认此入口是否符合预期，或改为独立 Splash/启动判断。
- \*\*`HUAWEI\_REDIRECT\_URI` 是否必填\*\***：`LoginWithHuaweiIDButton` 服务端换码通常不需要 `r  
\_uri`；若 AGC 配置要求回填, 则在 `.env` 补 `HUAWEI\_REDIRECT\_URI`。需用户在 AGC 操作后确认。
- \*\*`module.json5` metadata\*\***：仅当 AGC 的 Client ID ≠ App ID 时才需在 `module.json5  
`clientId` metadata；默认不改 (见文件列表注)。需用户提供 AGC 实际取值后确认。
- \*\*真机验证前置\*\***：Account Kit 在\*\*模拟器无法验证\*\*，须真机 + AGC 已完成 (创建应用、开通 Ac
- 6 指纹、取 Client ID/Secret)。此为用户侧前置依赖，非本任务可自测。
- \*\*P2 能力\*\***：账号绑定管理、华为手机号一键登录 (需申请 `quickLoginMobilePhone`)、埋点--本
- 评估。

---

架构设计完成，可转工程师。

// ===== ./docs/nav-migration-design.md =====  
# 跨 Tab 跳转修复 & Navigation 迁移评估

> 架构师: Bob | 针对问题: 「去发现」按钮跨 Tab 跳转失效 | 日期: 2025-07-11

---

## 1. 推荐方案与理由

### 方案对比

维度	方案 A (最小侵入)	方案 B (半迁移)	方案 C (全量迁移)
改动文件数	3	10+	15+
预计工时	0.5 天	3-5 天	5-8 天
编译风险	低	中高	高
回归风险	极低	中高	高

| 即时解决问题 |  |  |  |  
| 与未来迁移契合度 | 中性 (不阻碍) | 差 (混合态更难迁移) | 最佳 (一步到位) |

### 推荐：方案 A

**\*\*理由：\*\***

1. **\*\*问题明确，方案对症\*\***：「去发现」按钮失效的根因是 `AppStorage.setOrCreate('homeTabIndex')` 从未监听。方案 A 补上监听端即可闭合链路。
2. **\*\*已有验证模式\*\***：项目中 HomeTab 已用 `@StorageLink('authToken') @Watch('onTokenRead')` 实现登录态响应 (HomeTab.ets 第 39 行)，方案 A 完全复用此模式，无新技术引入。
3. **\*\*风险最低\*\***：仅改 3 个文件，不触碰 Tabs 结构、不引入 Navigation、不改路由体系。
4. **\*\*不阻碍未来迁移\*\***：方案 C (全量 Navigation 迁移) 应作为独立 epic 规划。方案 A 的 AppStorage.vPathStack 替换，不产生技术债。
5. **\*\*方案 B 不推荐\*\***：混合态 (Tabs 内 + Navigation 外) 导致两套导航范式共存，维护成本高于 A，最差。

### 长期迁移路径建议

...

----- 第 6 页 -----

当前 → 方案 A (立即修复，本次迭代) → 方案 C (独立 epic, 下个迭代)

方案 C 的迁移应在功能稳定后作为专项任务执行，包含：

- Index 改为 Navigation + 自定义 TabBar
- 8 个 `router.pushUrl` 页面迁移为 NavDestination
- 全量回归测试

---

## 2. 实现细节

### 涉及文件

文件	改动类型	说明
`entry/src/main/ets/utils/nav.ts`	新增函数	添加 `emitTabSwitch()` 信号发射器
`entry/src/main/ets/pages/Index.ets`	修改	添加 `@StorageLink` + `@Watch` 监听
`entry/src/main/ets/components/HomeTab.ets`	修改	替换「去发现」onClick 实现

### 改动点

#### 2.1 `utils/nav.ts` - 新增跨 Tab 跳转信号

在现有 `takeDetailPostId()` 之后追加：

```
````typescript
// 跨 Tab 跳转信号：子组件调用此函数触发 Index 切换 Tab。
// 用递增序列号保证 @Watch 每次都能触发 (@Watch 对同值不触发)。
export function emitTabSwitch(target: number): void {
  AppStorage.setOrCreate('tabSwitchTarget', target);
  const seq: number = (AppStorage.get<number>('tabSwitchSignal') ?? 0) + 1;
  AppStorage.setOrCreate('tabSwitchSignal', seq);
}
```

2.2 `Index.ets` - 添加监听

****新增属性**** (`@State currentIndex` 下方)：

```

```typescript
@StorageLink('tabSwitchSignal') @Watch('onTabSwitch') tabSwitchSignal: number =
0;
```

```

****新增方法**** (`refreshUnread()` 下方) :

```

```typescript
// 跨 Tab 跳转监听: 子组件 emitTabSwitch() 写入信号后触发
onTabSwitch(): void {
 const target: number = AppStorage.get<number>('tabSwitchTarget') ?? 0;

```

```

----- 第 7 页 -----
 if (target >= 0 && target <= 4) {
 this.currentIndex = target;
 }
}
```

```

2.3 `HomeTab.ets` - 修复 onClick

****第 11 行 import 修改**** (追加 `emitTabSwitch`) :

```

```typescript
import { setDetailPostId, emitTabSwitch } from '../utils/nav';
```

```

****第 354-359 行 onClick 替换****:

```

```typescript
Button('去发现')
 .margin({ top: 12 })
 .onClick(() => {
 emitTabSwitch(1); // 跳转到底部「圈子」Tab (CircleTab)
 })
```

```

数据流

```

```mermaid
sequenceDiagram
 participant U as 用户
 participant HT as HomeTab (Tab 0)
 participant NAV as utils/nav.ts
 participant AS as AppStorage
 participant IDX as Index (@Entry)
 participant T as Tabs 组件

 U->>HT: 点击「去发现」
 HT->>NAV: emitTabSwitch(1)
 NAV->>AS: setOrCreate('tabSwitchTarget', 1)
 NAV->>AS: setOrCreate('tabSwitchSignal', N+1)
 AS-->>IDX: @StorageLink 同步 tabSwitchSignal 变化
 IDX->>IDX: @Watch('onTabSwitch') 触发
 IDX->>AS: get('tabSwitchTarget') → 1
 IDX->>IDX: this.currentIndex = 1
 IDX->>T: index={this.currentIndex} 重渲染
 T-->>U: 切换到 CircleTab (Tab 1)
```

```

3. 任务列表

----- 第 8 页 -----

...

T1: 实现跨 Tab 跳转信号机制 - utils/nav.ts, Index.ets, HomeTab.ets - [依赖: 无]

> 仅 1 个任务, 3 个文件, 内聚度高。无前置依赖, 工程师可直接开工。

4. 共享知识

AppStorage key 命名约定

| Key | 类型 | 用途 | 写入方 | 读取方 |
|-----------------|--------|------------------|----------------------------|----------------------|
| tabSwitchSignal | number | 递增序列号, 触发 @Watch | emitTabSwitch() | Index @onTabSwitch() |
| tabSwitchTarget | number | 目标 Tab 索引 (0-4) | emitTabSwitch() | Index @onTabSwitch() |
| homeTabIndex | ~~ | - | **废弃** (原 HomeTab 写入但无人监听) | - |

@Watch 回调命名约定

- 格式: `on` + 被监听变量语义名, 如 `tabSwitchSignal` → `onTabSwitch`
- 与现有 `onTagChanged`、`onTokenReady` 风格一致
- 不标记 `private` (与现有 watch 回调一致)

边界条件处理

| 场景 | 行为 | 原因 |
|-----------------------|--|-------------|
| 连续点击「去发现」 | 每次递增 signal, 每次触发 watch, `currentIndex` 重复设为 1 | 无副作用, 不重复切换 |
| 已在圈子 Tab 时点击 | 不可能触发 (按钮仅在 HomeTab 关注流空态渲染) | - |
| signal 初始值 0 | 不触发 watch (@Watch 仅在值变化时触发) | 初始化不算变化 |
| target 未设置时 watch 误触发 | `?? 0` 兜底, 0 是合法 Tab 但不会误触发 (signal 仅由 emitTabSwitch 设置) | - |

5. 待明确事项

1. **未来是否有其他跨 Tab 跳转需求?** 当前 `emitTabSwitch(target)` 已支持任意目标 Tab。如首页 / MessagePage 需要跳转到首页, 直接调用 `emitTabSwitch(0)` 即可, 无需额外开发。
2. **方案 C 迁移排期:** 建议下个迭代作为独立 epic, 需 PM 确认优先级。当前方案 A 不阻碍后续迁移。
3. **`homeTabIndex` 旧 key 残留:** AppStorage 中可能残留旧值 `homeTabIndex`, 不影响功能, pStorage 非持久化, 重启后消失)。

6. 风险评估

编译风险

| 风险点 | 等级 | 说明 | 应对 |
|--------------------------|----|-------------------------------------|----|
| AppStorage.get<number>() | 低 | 项目中已有 `AppStorage.get<string>()` 先例 | |

----- 第 9 页 -----

| `AppStorage.get<number>()` 泛型用法 | 低 | 项目中已有 `AppStorage.get<string>()` 先例

.ets:14), number 同理 | `?? 0` 兜底 undefined |
| `AppStorage` 在 `.ts` 文件可用性 | 低 | `nav.ts` 已被 ArkTS 编译管线处理, `AppStorage`
象 | 若编译报错, 将函数移至 `.ets` 文件 |
| `@Watch` 回调非 private | 无 | 与现有 `onTagChanged` / `onTokenReady` 一致 | - |
| 新增 import 语句 | 无 | `emitTabSwitch` 从已有 `utils/nav.ts` 导入, 无新依赖 | - |

运行时风险

| 风险点 | 等级 | 说明 | 应对 |
|-----|-----|-----|-----|
| @Watch 在 @Entry 组件不触发 | 极低 | @StorageLink + @Watch 在 @Entry 中与子组件行为一致
已验证此模式 | QA 验证点击「去发现」后 Tab 切换 |
| @Watch 回调内读 AppStorage 时序 | 极低 | @Watch 在值变更后触发, 此时 `tabSwitchTarget`
witchSignal 写入 | emitTabSwitch 中 target 先写、signal 后写 |
| signal 递增值溢出 | 可忽略 | JS number 为 64 位浮点, 实际不可能溢出 | - |

回归风险

| 风险点 | 等级 | 说明 | 应对 |
|-----|-----|-----|-----|
| 普通 Tab 点按/滑动切换 | 无 | `onChange` 回调不变, Tabs 内置行为不受影响 | QA 回归 5 个 T
| 消息 Tab 红点 | 无 | `@StorageLink('unreadMessageCount')` 不受影响 | QA 回归未读红点 |
| 登录跳转 | 无 | `aboutToAppear` 中 `router.pushUrl` 不变 | QA 回归未登录→登录流程 |
| 其他页面跳转 | 无 | 不涉及 DetailPage / SearchResultPage 等路由 | - |

// ===== ./docs/system_design.md =====
大蓝书 · 帖子搜索功能 · 系统设计与任务分解

- > 角色: 交付架构师 (高见远)
- > 范围: P0 (搜索可用) + P1 (加载/空/失败/跳详情); P2 (热搜/联想/高亮) 不排入任务
- > 技术栈: 后端 Node.js + Express + TypeScript + Prisma + MySQL (端口 3000); 前端 Harmo
EXT 原生 (ArkTS + ArkUI, Stage 模型 **API 24**, ArkTS V1 严格模式)
- > 配套图: `docs/sequence-diagram.mermaid` (调用时序)、`docs/class-diagram.mermaid` (类

0. 已核事实与对 PRD 假设的必要修正 (重要, 请先读)

我直接阅读了现有代码, 发现 PRD/任务说明中的几处描述与**实际代码**不符。以下为权威事实, 后续设计以

| # | PRD 原假设 | 实际代码 (已核实) | 对本次设计的影响 |
|---|---|---|---|
| F1 | `tags(String[])`、`images(String[])` | `prisma/schema.prisma`: `tags Json`、
`images Json` | 标签匹配用 Prisma `arrayContains` (Json 数组精确包含), **不是** `has`。
不参与搜索。 |
| F2 | `listPosts` 返回 `{items,total,page,pageSize}`, 且经 `toPostDTO()` 序列化 | `po
Service.ts` 实际返回 `{ list, pagination: {page, limit, total} }`, \*\*无\*\* `toPostDTO()
, 直接返回 Prisma 对象 (含 `user` 关联) | `searchPosts` 必须返回**同形** `ListResult<Po
list, pagination`。前端 `api.ets` 已定义 `ListResult<T> = {list, pagination}`, 直接复
| F3 | `PostCard` 自带点击 emit 跳详情 | `PostCard.ets` **无**点击处理; 点击由 `HomeTab`
Item().onClick 父级包裹实现 (写 `AppStorage` + `setDetailPostId` + `router.pushUrl`
(`es/DetailPage`)) | `SearchResultPage` **照抄** `HomeTab` 的父级包裹点击模式即可, **不要
ard**。
| F4 | `main\_pages.json` 在 `entry/src/main/ets/` | 实际路径 `entry/src/main/resourc
s/base/profile/main_pages.json` | 注册 `SearchResultPage` 改这个文件。 |
| F5 | `router.getParams()` 传参 | `utils/nav.ts` 注释明确: "router.getParams() 在 AP
4 已弃用且返回空" | keyword 跨页传递用 `AppStorage.setOrCreate('searchKeyword', kw)`, 与
ilPostId` 既有做法一致。 |

1. 实现方案 + 框架选型

****结论：完全沿用现有栈，仅做最小增量改动，不引入新框架/新依赖。****

----- 第 10 页 -----

1.1 后端：扩展 `GET /v1/posts`，新增可选 `keyword` 查询参数

- 不新增独立端点 (PRD 决策 #5)，复用 `listPosts` 的分页/排序管线与返回结构，最小化改动面。
- `listPosts(params)` 新增可选 `keyword?: string`；当 `keyword` 非空时，在现有 `where` s: 1 + 可选 tag/author) 基础上叠加 `OR` 条件，命中**任一**字段即返回 (PRD 决策 #1 用 OR)。
- 四个命中字段及匹配方式 (基于 F1 的 Json 现实)：
 - `title` → `{ contains: kw, mode: 'insensitive' }`
 - `content` → `{ contains: kw, mode: 'insensitive' }` (content 可空，空值自然不匹配)
 - `genre` → `{ contains: kw, mode: 'insensitive' }`
 - `tags` (Json) → `{ arrayContains: kw }` (**精确包含某标签**，非子串模糊；Json 列无法用串模糊，且非 v1 范围，PRD 决策 #2 仅要求 LIKE 级模糊，标题/正文/体裁已满足)

```
``ts
// backend/src/services/postService.ts -- listPosts 内新增
if (params.keyword) {
  const kw = String(params.keyword).trim();
  if (kw) {
    where.OR = [
      { title: { contains: kw, mode: 'insensitive' } },
      { content: { contains: kw, mode: 'insensitive' } },
      { genre: { contains: kw, mode: 'insensitive' } },
      { tags: { arrayContains: kw } },
    ];
  }
}
```

> 说明：`where` 类型为 `any`；Prisma 在顶层 `where` 中会把 `status` 与 `OR` 自动 AND 处理已发布 且 (标题|正文|体裁|标签 命中关键词)"，符合预期。`mode: 'insensitive'` 需 **MySQL 8.0**)。

1.2 排序与分页

- 排序：默认按 `createdAt desc` (`latest`)；`sort` 参数预留 `latest` | `hot`，本期仅实现 `latest`。
- 分支 (PRD 决策 #3)。后端 `listPosts` 已有 `sort` 分支，无需改动排序逻辑。
- 分页：`page` + `limit`，后端已有 `skip/take` 与 `count`。前端 `searchPosts` 默认 `page` 10 (PRD 决策 #4)；结果页**滚动到底加载更多**，不显示分页按钮。

1.3 前端

- `api.ets` 新增 `searchPosts(params: SearchPostsParams)`：复用 `toQuery` 拼 `keyword/page/limit/sort`，请求 `GET /v1/posts`。
- `HomeTab` 搜索框 `onSubmit`：由 `promptAction.showToast` 占位改为「写 `AppStorage` + `enter.pushUrl('pages/SearchResultPage')`」。
- 新建 `SearchResultPage`：自有顶栏 (返回 / 可改搜索框 / 清除 x / 结果计数)，复用 `PostCard` 渲染，`fresh` 下拉刷新 + `onScrollIndex` 上拉加载更多 + 三态 (加载/空/失败重试)，点击跳详情沿用 F3。

2. 文件列表 (标注 新增 / 修改)

后端 (`backend/src`)

| 文件 | 操作 | 改动点 |
|--|-------------|---|
| ----- ----- ----- | | |
| `services/postService.ts` | **修改** | `ListParams` 增加 `keyword?`；`listPosts` 内加 OR 过滤 (见 §1.1)。 |
| `routes/posts.ts` | **修改** | `GET /` 解构并透传 `keyword` 查询参数给 `listPosts`。 |
| `__tests__/postService.search.test.ts` | **新增 (可选)** | 对 `listPosts({keyword})` 的 OR 命中做单测 (标题/正文/体裁/标签各一例 + 空结果)。不强制，建议补。 |

前端 (`entry/src/main/ets`)

| 文件 | 操作 | 改动点 |
|-----|-----|-----|

----- 第 11 页 -----

| `services/api.ets` | **修改** | 新增 `SearchPostsParams` 接口与 `searchPosts()`；复用
oQuery` 与 `ListResult<Post>`。 |
| `components/HomeTab.ets` | **修改** | 搜索框 `onSubmit`：写 `AppStorage('searchKeyw
d')` 并 `router.pushUrl('pages/SearchResultPage')` (替换 toast)。 |
| `pages/SearchResultPage.ets` | **新增** | 结果页：顶栏 + 2 列 `PostCard` 网格 + 下拉刷
加载 + 三态 (加载/空/失败重试) + 点击跳详情。 |
| `../resources/base/profile/main\_pages.json` (路径见 F4) | **修改** | `src` 数组追加
ges/SearchResultPage"。 |

> 说明：`SearchResultPage.ets` 仅**复用** `PostCard` (`@Prop post`)，不修改 `PostCard.
F3)。

3. 数据结构和接口

见 `docs/class-diagram.mermaid`。核心契约如下 (ArkTS / TS 表达)：

3.1 前端入参 (新增)

```
``ts
// services/api.ets
export interface SearchPostsParams {
  keyword: string;
  page?: number; // 默认 1
  pageSize?: number; // 默认 10 (= 后端 limit)
  sort?: SortType; // 'latest' | 'hot', 本期仅 latest 生效
}
```

3.2 复用数据结构 (models/types.ets / api.ets)

```
``ts
export interface Pagination { page: number; limit: number; total: number; }
export interface ListResult<T> { list: T[]; pagination: Pagination; }
// Post / User 见 models/types.ets (Post.user 可选, 由后端 user 关联填充)
```

3.3 接口形态 (扩展现有 GET /v1/posts)

GET /v1/posts?keyword=<kw>&page=1&limit=10&sort=latest

- 请求：`keyword` 必填 (前端保证非空再跳)，`page`/`limit`/`sort` 可选。
- 响应 (与现有列表**完全一致**，F2)：

```
``json
{ "code": 0, "message": "ok",
  "data": {
    "list": [ { "id": 1, "title": "...", "content": "...", "genre": "review",
               "tags": ["数码选购"], "images": [...], "upCount": 12,
               "bookmarkCount": 3, "commentCount": 5,
               "createdAt": "2025-07-01T08:00:00.000Z",
               "user": { "id": 9, "nickname": "老王", "avatar": "..." } } ],
    "pagination": { "page": 1, "limit": 10, "total": 37 }
  }
}
```

- 错误：`code !== 0` 或 HTTP 非 200 → 前端进入失败态 (P1-3 重试)。

3.4 后端 ListParams (修改后)

```
```.ts
// services/postService.ts
export interface ListParams {
 page?: number;
 limit?: number;
 sort?: SortType;
 tag?: string;
 author?: number;
 keyword?: string; // 新增
}
```

---

## ## 4. 程序调用流程 (时序图)

完整时序见 `docs/sequence-diagram.mermaid`, 要点:

1. **触发**: `HomeTab` 搜索框 `onSubmit` → `AppStorage.setOrCreate('searchKeyword', keyword)` → `router.pushUrl({url: 'pages/SearchResultPage'})`。
2. **初始化**: `SearchResultPage.aboutToAppear` 从 `AppStorage` 读 `searchKeyword` → `refresh()`。
3. **首屏**: `searchPosts({keyword, page:1, limit:10, sort:'latest'})` → `GET /v1/posts?keyword=..&page=1&limit=10&sort=latest` → `PostsRoute` → `postService.listPosts({keyword,...})` → `prisma.post.findMany({where:{status:1, OR:[...]}, orderBy, skip, take})` + `count` → 返回 `{list, pagination}` → 渲染 2 列 `PostCard` 网格 + 找到 N 篇。
4. **加载更多**: `Grid.onScrollIndex` 末项可见 → `loadMore()` (`page++`) → 再次 `search` → `posts.concat(res.list)`; 当返回条数 `< limit` 置 `finished`。
5. **点击跳详情**: 父级 `GridItem.onClick` → `AppStorage.setOrCreate('detailPostId', id)` + `setDetailPostId(id)` + `router.pushUrl('pages/DetailPage')` (同 F3)。
6. **失败重试**: `fetch` 抛错 → `error` 置位 → 空态区显示 "加载失败: ..." + **重试按钮** → `fetch`。

---

## ## 5. 任务列表 (有序、含依赖、按实现顺序)

> 按依赖顺序执行; P0 必须全部完成方可联调, P1 为体验增强但建议同批完成。

任务	名称	源文件	依赖	优先级
***T1**	后端 `listPosts` 增加 keyword OR 过滤	`services/postService.ts` (改); `ts_/postService.search.test.ts` (新增, 可选)	无	P0
***T2**	后端路由透传 `keyword` 查询参数	`routes/posts.ts` (改)	T1	P0
***T3**	前端 `api.ets` 新增 `searchPosts`	`services/api.ets` (改)	T1, T2	P0
***T4**	`HomeTab` 搜索框 `onSubmit` 改跳转	`components/HomeTab.ets` (改)	T3	
***T5**	新建 `SearchResultPage` (2列网格 + 分页 + 三态 + 跳详情)	`pages/SearchResultPage.ets` (新增)	T3, T4	P0(P0-4) + P1(P1-1~P1-4)
***T6**	注册 `SearchResultPage` 路由	`resources/base/profile/main_pages.json` (改)	T5	P0
***T7**	联调与三态验证 (含失败重试、空态、加载更多)	上述全部	T1-T6	P0+P1 收尾

依赖链: `T1 → T2 → T3 → {T4, T5} → T6 → T7`。T4 与 T5 可并行 (都只依赖 T3)。

---

## ## 6. 依赖包列表

**\*\*本期无需新增任何 npm 依赖，也无需新增 ohpm 依赖。\*\***

----- 第 13 页 -----

- 后端：仅用既有 Prisma ORM 条件拼接 + Express，无新包。
- 前端：仅复用既有 `@kit.ArkUI` / `@kit.NetworkKit` / `AppStorage`，无新包。

---

## ## 7. 共享知识 (跨文件约定, Engineer 必读)

1. **\*\*ArkTS V1 严格模式\*\***: `@State` 禁用保留字 (`tabIndex` / `width` / `height` 等); **\*\*禁止解量必须对应具名 `interface`**; `http` 请求体走 `extraData: JSON.stringify(body)` (本项目 `r` 已封装, 勿改)。`SearchResultPage` 所有状态用显式 `@State` 标量/数组, 不依赖解构。
2. **\*\*跨页传参用 `AppStorage`\*\***: `router.getParams()` 在 API 24 失效 (F5)。`HomeTab` 跳转 `AppStorage.setOrCreate('searchKeyword', kw)`; `SearchResultPage.aboutToAppear` 用 `AppStorage.get('searchKeyword')` 读取并初始化。详情跳转为 `AppStorage.setOrCreate('detailId', id)` + `setDetailPostId(id)` (沿用既有 `utils/nav.ts`)。
3. **\*\*`PostCard` 不改, 点击由父级包裹\*\***: `SearchResultPage` 直接复用 `PostCard({post})` 模板上 `onClick` 写 `detailPostId` 并 `router.pushUrl('pages/DetailPage')` , **\*\*完全照 `HomeTab` 第 181-191 行模式\*\***。
4. **\*\*分页 / 三态 UI 模式照抄 `HomeTab`\*\***: `Refresh` 下拉刷新 + `Grid.onScrollIndex(end>.length-1)` 上拉加载更多 + 状态变量 `loading/finished/isRefreshing/error`。区别仅在**\*\*失败按钮\*\*** (P1-3), 其余 (加载中 `LoadingProgress`、空态文案) 与 `HomeTab` 视觉一致。
5. **\*\*返回结构约定 (重要, 基于 F2)\*\***: `searchPosts` 与 `listPosts` 同形返回 `ListResult<Post, pagination:{page, limit, total}>` , **\*\*无 `toPostDTO`\*\***; `finished` 判定用 `res.length < this.limit` (不要用写死的 20, 除非你特意把 `limit` 设为 20)。
6. **\*\*标签匹配用 `arrayContains` (基于 F1)\*\***: `tags` 是 `Json` 列, 关键词匹配标签用 `{ tag, contains: kw }` (精确包含某标签); 不做数组元素子串模糊 (Json + ORM 不支持且无 v1 必要)。首 `Contains` , 保持统一。

---

## ## 8. 待明确事项

1. **\*\*搜索结果页接收 keyword 的方式\*\***: 已据 API 24 实测 (`utils/nav.ts` 注释) 决定用 `AppStorage` 已落地于共享知识②, 无需再确认。
2. **\*\*`searchPosts` 默认 `pageSize`\*\***: PRD 决策 #4 定为 **\*\*10\*\***; 而 `HomeTab` 列表用 `limit` 0。两者不一致但是 PRD 有意为之 (搜索结果页默认 10)。若希望与首页视觉节奏统一, 可把 `searchPosts` 0 -- **\*\*默认按 PRD 用 10\*\***, 如团队偏好 20 请联调前告知。
3. **\*\*`mode: 'insensitive'` 数据库前提\*\***: 需 MySQL 8.0+ (本地库默认满足)。若部署库为 MySQL 8.0 以下, 需 `mode: 'insensitive'` (中文搜索大小写不敏感无影响)。
4. **\*\*既有 `HomeTab` 标签筛选\*\***: 其 `postService.ts` 第 21 行使用 `arrayContains` (与 `Json` 一致, **\*\*正确\*\***), 本次 keyword 的 tags 匹配同样用 `arrayContains` , 保持统一, 无需改动既有。
5. **\*\*其余 PRD 待确认项\*\*** (匹配 OR 逻辑、LIKE 方案、默认排序、滚动分页、扩展 GET /v1/posts、按默认决策落地, 本期不排 P2 (热搜/联想/高亮))。

---

## ## 9. 任务依赖图 (Mermaid)

```
``mermaid
graph TD
 T1[T1 后端 listPosts keyword OR 过滤] --> T2[T2 路由透传 keyword]
 T2 --> T3[T3 前端 api.ets searchPosts]
 T3 --> T4[T4 HomeTab onSubmit 跳转]
 T3 --> T5[T5 新建 SearchResultPage 网格+三态]
 T4 --> T6[T6 注册路由 main_pages.json]
 T5 --> T6
 T6 --> T7[T7 联调与三态验证]
```

...

```
// ===== ./entry/Docs/design-account-binding.md =====
系统设计文档 · 账号绑定管理页 (大蓝书)
```

```
> 文档类型: 架构设计 + 任务分解 (不含实现代码)
> 作者: 高见远 (架构师)
> 关联 PRD: `entry/Docs/prd-account-binding.md`
> 技术栈: 前端 HarmonyOS NEXT (ArkTS + ArkUI, API 24, V1 严格模式) · 后端 Node.js + Express
pScript + Prisma + MySQL (端口 3000)
```

---

```
----- 第 14 页 -----
0. 核心设计决策 (拍板项速览)
```

```
| 决策点 | 结论 |
| :--- | :--- |
| **数据模型** | 采用**独立 `UserBinding` 表** `{ id, userId, provider, externalId, bo
At, isPrimary, extra? }`, `(userId, provider)` 唯一。 |
| **`User.unionID` 冗余列** | **保留并同步** `UserBinding` 与 `User.uni
有 `loginWithHuawei` (按 `unionID` findOrCreate) 逻辑。 |
| **鸿蒙主账号在列表中的呈现** | **不落 `UserBinding` 行** `GET` 接口从 `User.openId`
eatedAt` **合成**为 `provider='harmony'` 的置顶项 (`isPrimary=true`)。理由: 主账号是账号
不属于"绑定关系"语义; 这样既满足"列表以 `UserBinding` 为准" (第三方关系唯一真源仍是表), 又无需
| **`provider` 枚举值** | `'harmony'` (鸿蒙主账号) / `'huawei'` (华为) / `'wechat'` (微信,
义, 避免散落字符串。 |
| **华为授权交互** | 走 Account Kit **系统级授权页** (复用 `LoginWithHuaweiIDButton` 思路
eiBindButton` 暴露 `unionID`), 非 WebView。前端直接取 `unionID` 调 `POST /v1/account/b
gs` (body `{provider, externalId}`), 契合既定接口契约。 |
| **解绑策略** | 鸿蒙主账号**无解绑入口 + DELETE 接口直接 403** `第三方 (华为) 自由解绑。主账号
约束。 |
| **Prisma 变更方式** | 用 `prisma db push` (非 migrate), 因 DB 用户无建库权; 变更仅为**新
(对 `User` 无新增列), 属安全可加操作。 |
```

---

```
1. 实现方案 + 框架选型
```

```
1.1 后端落地
```

```
- **框架**: 沿用现有 Express + TS + Prisma + MySQL, 端口 3000, 统一响应 `{ code, data, m
}`。
- **鉴权中间件复用**: 直接 `import { auth } from '../middleware/auth'`, 三个接口全部 `aut
eq.userId` 来自 JWT `payload.userId`, **所有 DB 操作强制以 `req.userId` 为过滤条件**, 杜绝
- **路由组织**: 新增 `backend/src/routes/account.ts`, 在 `backend/src/app.ts` 中以 `app
('/v1/account', accountRouter)` 挂载 (与现有 `/v1/auth`、`/v1/posts` 同级)。
- **业务分层**: `account.ts` 仅做参数校验 + 鉴权透传 + 调 `accountBindingService.ts`; 占月
同步 `User.unionID` / 脱敏 全部在 service 内, 便于单测 (复用现有 `auth.huawei.test.ts` 的
ss 测试模式)。
- **错误码**: **完全复用** `utils/response.ts` 现有 `CODE` (`401/400/409/403/404/500`)
改动 `response.ts`。
- **Schema 变更方式**: `npx prisma db push` 推送 schema 到 DB (不生成迁移文件); 随后 `npx
nerate` 刷新客户端类型。新增 `UserBinding` 为独立新表, 并在 `User` 上追加虚拟关系 `bindings`
(无新列, 仅外键落在 `UserBinding.userId`), 属纯增量、零数据风险操作。
```

```
1.2 前端落地
```

```
- **语言/框架**: ArkTS + ArkUI, V1 严格模式 (catch 不标类型、throw 只抛 `Error`、对象字面量
- **请求封装复用**: 在现有 `services/api.ets` 的 `api` 客户端上**新增** `listBindings` / b
nt / unbindAccount` 三个函数 (不新建文件, 保持单 API 文件结构)。
- **鉴权头**: `api.ets` 的 `request` 已自动从 `AppStorage.get('authToken')` 注入 `Autho
tion: Bearer <token>`, 无需改动。
```

- **\*\*华为授权\*\***: 新建 `components/HuaweiBindButton.ets` , 复用 `@kit.AccountKit` 的 `LoginWithHuaweiIDButton` + `loginComponentManager` , 回调提取 `response.unionID` 后通过 `onHuaweiBindSuccess(unionID)` 暴露给页面 (与现有 `HuaweiLoginButton` 只取 `code` 形成职责区分, 互斥)  
- **\*\*提示\*\***: 统一用 `utils/toast.ets` 的 `safeShowToast(this.getUIContext(), { message: ... })`  
- **\*\*导航\*\***: 沿用 `router.pushUrl` / `router.back()` (本期不迁 Navigation) 。新页面需在 `resources/base/profile/main\_pages.json` 注册。  
- **\*\*入口改造\*\***: `ProfilePage.ets` 右上角"设置"由占位 toast 改为 `router.pushUrl({ url: 'pages/SettingsPage' })` ; `SettingsPage.ets` 放"账号绑定"行跳 `AccountBindingPage.ets` 。

### ### 1.3 技术难点与选型理由

- **\*\*难点 1 · unionID 占用校验跨两套存储\*\***: 华为 unionID 可能存在于 `UserBinding` (新) 或 `User` (旧登录) 。→ 校验时**\*\*两处都查\*\***, 任一命中他人即返回 `409` 。  
- **\*\*难点 2 · 主账号永驻且不可解绑\*\***: → 列表合成 + DELETE 接口对 `harmony` 直接 `403` , 双保险  
- **\*\*难点 3 · ArkTS V1 严格模式限制\*\***: → 对象字面量 (接口请求体、BindingItem) 全部带显式类型; 避免 `error` 。  
- **\*\*选型理由\*\***: 独立 `UserBinding` 表契合"列出全部绑定 / 增删 / 统计 / 横向扩 provider"的诉求; `provider` 只插一行不改 schema; 冗余 `User.unionID` 兼容既有华为登录链路, 成本极低。

---

## ## 2. 文件列表及相对路径

### ### 2.1 后端 (相对 `backend/` )

操作	路径	说明
:	---	---

----- 第 15 页 -----

改	`prisma/schema.prisma`	新增 `UserBinding` model; `User` 增加 `bindings` 关系
改	`src/app.ts`	挂载 `app.use('/v1/account', accountRouter)`
新	`src/routes/account.ts`	三个接口路由 (GET/POST/DELETE `/v1/account/bindings` )
用	`auth` 中间件	
新	`src/services/accountBindingService.ts`	占用校验、写表、同步 `User.unionID` 、形成
新	`src/routes/account.test.ts`	in-process 集成测试 (复用 `auth.huawei.test.ts` , mock prisma)

> 注: `src/utils/response.ts` 、 `src/middleware/auth.ts` 、 `src/prisma.ts` **\*\*无需改动\*\*** (码与鉴权已齐备) 。

### ### 2.2 前端 (相对 `entry/` )

操作	路径	说明
:	---	---

改	`src/main/ets/models/types.ets`	新增 `Provider` 、 `BindingItem` 类型与请求/响应
改	`src/main/ets/services/api.ets`	新增 `listBindings` / `bindAccount` / `unbindAccount` 及入参类型
新	`src/main/ets/components/HuaweiBindButton.ets`	复用 `LoginWithHuaweiIDButton` 暴露 `unionID` 的绑定专用按钮
新	`src/main/ets/pages/SettingsPage.ets`	极简设置页, 含"账号绑定"入口行
新	`src/main/ets/pages/AccountBindingPage.ets`	账号绑定管理页 (列表 + 绑定/解绑 + 绑定/解绑)
改	`src/main/ets/pages/ProfilePage.ets`	"设置"由占位 toast 改为跳 `SettingsPage`
改	`src/main/resources/base/profile/main_pages.json`	注册 `pages/SettingsPage` 、 `pages/AccountBindingPage`

> 注: `utils/auth.ets` 、 `utils/toast.ets` 、 `utils/nav.ts` **\*\*无需改动\*\*** (登录态、toast 已

---



## ## 3. 数据结构和接口

### ### 3.1 Prisma Model (追加到 `schema.prisma`)

```
``prisma
// 账号绑定关系表 (第三方/主账号绑定统一以本表为准, 横向扩展友好)
model UserBinding {
 id Int @id @default(autoincrement())
 userId Int
 user User @relation(fields: [userId], references: [id], onDelete: Cascade)
 provider String @db.VarChar(32) // 'harmony' | 'huawei' | 'wechat' ... 见共享知识
 externalId String @db.VarChar(128) // 各 provider 的外部标识: openId / unionID / openid
 boundAt DateTime @default(now())
 isPrimary Boolean @default(false) // 当前仅 harmony 主账号为 true (合成项)
 extra Json? // 扩展位: 头像/昵称快照等, 本期未用

 @@unique([userId, provider]) // (userId, provider) 唯一: 同一用户同 provider 仅一条
 @@index([provider, externalId]) // 占用校验: 快速查"该 externalId 是否已被他人占用"
}

model User {
 // ... 现有字段保持不变 (id / openId / unionID / nickname / avatar / ...) ...
 bindings UserBinding[] // ← 新增虚拟关系 (不新增列, FK 在 UserBinding)
}
}...
```

----- 第 16 页 -----

### ### 3.2 前端 `BindingItem` 接口 (追加到 `types.ets`)

```
``ts
// 绑定 provider 枚举 (与后端保持一致, 单一来源)
export type Provider = 'harmony' | 'huawei' | 'wechat';

// 账号绑定列表项 (后端返回已脱敏 externalId)
export interface BindingItem {
 provider: Provider; // 'harmony' | 'huawei' | 'wechat'
 externalId: string; // 已脱敏展示串, 如 '****1234'
 boundAt: string; // ISO 时间串
 isPrimary: boolean; // true 仅鸿蒙主账号
 displayName: string; // 中文展示名: '鸿蒙账号' / '华为账号' / '微信'
 status: 'primary' | 'bound' | 'unbound'; // 主账号 / 已绑定 / 未绑定 (P1 预留)
}

// 绑定请求体 (POST /v1/account/bindings)
export interface BindAccountBody {
 provider: Provider;
 externalId: string; // 华为场景即 unionID
}

// 解绑响应 (DELETE /v1/account/bindings/:provider)
export interface UnbindResult {
 provider: Provider;
 unbound: boolean;
}
}...
```

### ### 3.3 三个后端接口契约

**\*\*GET `/v1/account/bindings`\*\* (需鉴权)**

- 响应 `data`: `BindingItem[]`, 固定顺序: 鸿蒙 (主账号, 合成) → 华为 → 微信 (P1, 本期无) → 其他
- 鸿蒙项由 `User.openId` / `User.createdAt` 合成, `isPrimary=true`, `externalId` 脱敏。

**\*\*POST `/v1/account/bindings`\*\* (需鉴权)**

- 请求体: `{ "provider": "huawei", "externalId": "<unionID>" }`
- 校验: provider 合法; externalId 非空; `harmony` 不允许主动绑定 (400)。
- 占用校验: 在 `UserBinding` (`provider+externalId`) 与 `User.unionID` 两处查"他人占用"。
- 成功: 写 `UserBinding` (`isPrimary=false`) + 同步 `User.unionID`; 响应 `data`: 新建的 `gItem`。

**\*\*DELETE `/v1/account/bindings/:provider`\*\* (需鉴权)**

- `:provider` 路径参数 (如 `huawei`)。
- 校验: `harmony` → `403` (主账号不可解绑); 不存在 → `404`。
- 成功: 删除该 `(userId, provider)` 行; 响应 `data`: `{ provider, unbound: true }`。

统一错误码: `401` (缺/过期 token, auth 中间件)、`400` (参数缺)、`409` (已被他人占用)、`403` (存在)、`500` (服务端)。

---

----- 第 17 页 -----

## ## 4. 程序调用流程 (时序图)

### ### 4.1 绑定华为账号

```
sequenceDiagram
 actor U as 用户
 participant ABP as AccountBindingPage
 participant HB as HuaweiBindButton(@kit.AccountKit)
 participant API as api.ets
 participant AUTH as auth 中间件
 participant SVC as AccountBindingService
 participant DB as Prisma / MySQL

 U->>ABP: 点击「绑定华为账号」
 ABP->>HB: 触发系统级授权页
 HB->>HB: LoginWithHuaweiIDButton 拉起
 HB-->>ABP: onHuaweiBindSuccess(unionID)
 ABP->>API: bindAccount('huawei', unionID)
 API->>AUTH: GET 带 Authorization: Bearer <token>
 AUTH-->>API: req.userId (JWT 解析)
 API->>SVC: bind(userId, 'huawei', unionID)
 SVC->>DB: 占用校验 (UserBinding + User.unionID, 排除自己)
 alt 已被他人占用
 DB-->>SVC: 命中
 SVC-->>API: throw 409 CONFLICT
 API-->>ABP: Error('该华为账号已关联其他大蓝书账号')
 ABP->>ABP: safeShowToast(错误提示)
 else 通过
 SVC->>DB: 写 UserBinding + 同步 User.unionID
 DB-->>SVC: BindingItem
 SVC-->>API: ok(BindingItem 已脱敏)
 API-->>ABP: BindingItem
 ABP->>ABP: 刷新列表 + safeShowToast('绑定成功')
 end
 ...
```

### ### 4.2 解绑第三方账号

```
```mermaid
sequenceDiagram
    actor U as 用户
    participant ABP as AccountBindingPage
    participant API as api.ets
    participant AUTH as auth 中间件
    participant SVC as AccountBindingService
    participant DB as Prisma / MySQL
```

```
U->>ABP: 点击「解绑华为」
ABP->>ABP: AlertDialog 二次确认
```

----- 第 18 页 -----

```
U->>ABP: 点「确定解绑」
ABP->>API: unbindAccount('huawei')
API->>AUTH: DELETE 带 Authorization: Bearer <token>
AUTH-->>API: req.userId
API->>SVC: unbind(userId, 'huawei')
SVC->>DB: 删 UserBinding (WHERE userId & provider)
DB-->>SVC: 删除行
SVC-->>API: ok({ provider:'huawei', unbound:true })
API-->>ABP: 成功
ABP->>ABP: 刷新列表 + safeShowToast('解绑成功')
Note over ABP: 鸿蒙主账号行无解绑按钮; 即便误调 DELETE /harmony, 接口直接 403
```

4.3 类 / 结构关系图

```
```mermaid
classDiagram
 class User {
 +Int id
 +String? openId
 +String? unionID
 +bindings UserBinding[]
 }
 class UserBinding {
 +Int id
 +Int userId
 +String provider
 +String externalId
 +DateTime boundAt
 +Boolean isPrimary
 +Json? extra
 }
 User "1" --> "0..*" UserBinding : bindings

 class AccountBindingService {
 +listBindings(userId) BindingItem[]
 +bind(userId, provider, externalId) BindingItem
 +unbind(userId, provider) UnbindResult
 -checkOccupied(provider, externalId, userId) boolean
 -maskExternalId(provider, raw) string
 }
 AccountBindingService ..> UserBinding : reads / writes

 class AccountRouter {
 +GET /v1/account/bindings
 +POST /v1/account/bindings
 }
```

```

+DELETE /v1/account/bindings/:provider
}
AccountRouter ..> AccountBindingService : calls
AccountRouter ..> AuthMiddleware : protects

```

----- 第 19 页 -----

```

class BindingItem {
+Provider provider
+string externalId
+string boundAt
+boolean isPrimary
+string displayName
+string status
}
AccountBindingService ..> BindingItem : returns

class HuaweiBindButton {
+onHuaweiBindSuccess(unionID) void
+onHuaweiBindFailure(error) void
}
class AccountBindingPage {
+BindingItem[] bindings
+loadBindings() void
+handleBindHuawei(unionID) void
+handleUnbind(provider) void
}
class SettingsPage
class ProfilePage
class ApiClient {
+listBindings() BindingItem[]
+bindAccount(provider, externalId) BindingItem
+unbindAccount(provider) UnbindResult
}

AccountBindingPage ..> HuaweiBindButton : uses
AccountBindingPage ..> ApiClient : calls
ApiClient ..> AccountRouter : HTTP /v1/account/bindings
SettingsPage ..> AccountBindingPage : pushUrl
ProfilePage ..> SettingsPage : pushUrl
...

```

---

## ## 5. 任务列表 (有序、含依赖、按实现顺序)

> 总任务数: \*\*4 个\*\* (后端 1 + 前端 2 + 联调 1)。后端接口先于前端联调。

### T01 · 后端: 数据模型 + 绑定服务 + 路由 (含挂载) 【P0, 无前置依赖】

- \*\*涉及文件\*\*:

- `backend/prisma/schema.prisma` (新增 `UserBinding` model + `User.bindings` 关系)
- `backend/src/app.ts` (挂载 `/v1/account` 路由)
- `backend/src/services/accountBindingService.ts` (新建: list/bind/unbind + 占用校验同步 unionID + 脱敏)
- `backend/src/routes/account.ts` (新建: 三个接口, 复用 `auth` 中间件)
- \*\*做什么\*\*:

----- 第 20 页 -----

1. 在 schema 中追加 `UserBinding` 与 `User.bindings` 关系; 执行 `npx prisma db push`  
`npx prisma generate`。
2. `app.ts` 增加 `import accountRouter` 与 `app.use('/v1/account', accountRouter)`

- 3. service 实现: 列表合成 (harmony 置顶 + 第三方按优先级)、绑定 (双处占用校验 → 409、写表、解绑 (harmony → 403、不存在 → 404、删除行)、maskExternalId`。
- 4. route 实现: 参数校验、调 service、用 `ok/fail` + `CODE` 统一响应。
- **\*\*验收点\*\*:**
  - `prisma db push` 成功, `UserBinding` 表存在, `(userId, provider)` 唯一约束生效。
  - 无 `Authorization` 调三接口均返回 `401`。
  - POST 同一 unionID 到两个不同账号: 第二个返回 `409`, `message` 提示"已关联其他账号"。
  - DELETE `/harmony` 返回 `403`; DELETE 不存在的 provider 返回 `404`。
  - GET 返回首项为 `provider='harmony'`、`isPrimary=true`, `externalId` 已脱敏。

### ### T02 · 前端: 数据模型 + API 封装 + 绑定按钮 【P0, 依赖 T01 接口契约】

- **\*\*涉及文件\*\*:**
  - `entry/src/main/ets/models/types.ets` (新增 `Provider`、`BindingItem`、`BindAccountBody`、`UnbindResult`)
  - `entry/src/main/ets/services/api.ets` (新增 `listBindings` / `bindAccount` / `unbindAccount`)
  - `entry/src/main/ets/components/HuaweiBindButton.ets` (新建: 复用 `LoginWithHuaweiBindButton` 暴露 `unionID`)
- **\*\*做什么\*\*:**
  1. `types.ets` 增加上述类型 (带显式上下文类型, 满足 V1 严格模式)。
  2. `api.ets` 增加三个函数, 复用现有 `api` 客户端 (自动带 token); `bindAccount` 抛错时由顶层捕获。
  3. `HuaweiBindButton.ets`: 用 `loginComponentManager` 回调提取 `response.unionID`, 暴露 `onHuaweiBindSuccess(unionID)` 暴露; 失败/取消走 `onHuaweiBindFailure`。
- **\*\*验收点\*\*:**
  - `api.ets` 三函数编译通过, 请求路径/方法/body 与 T01 契约一致。
  - `HuaweiBindButton` 点击能拉起系统级华为授权页 (非 WebView), 确认后回调拿到 `unionID`;
  - 类型定义与后端 `BindingItem` 字段对齐。

### ### T03 · 前端: 设置页 + 账号绑定页 + 入口改造 + 页面注册 【P0, 依赖 T02】

- **\*\*涉及文件\*\*:**
  - `entry/src/main/ets/pages/SettingsPage.ets` (新建: 极简设置页 + "账号绑定"行)
  - `entry/src/main/ets/pages/AccountBindingPage.ets` (新建: 列表 + 绑定/解绑 + 二次确认)
  - `entry/src/main/ets/pages/ProfilePage.ets` ("设置"改为跳 `SettingsPage`)
  - `entry/src/main/resources/base/profile/main\_pages.json` (注册两个新页面)
- **\*\*做什么\*\*:**
  1. `ProfilePage` 右上"设置" `onClick` 改为 `router.pushUrl({ url: 'pages/SettingsPage' })` (移除占位 toast)。
  2. `SettingsPage`: 顶栏返回 + "设置"标题, 列表项"账号绑定" → `router.pushUrl({ url: 'pages/AccountBindingPage' })`。
  3. `AccountBindingPage`: `aboutToAppear` 调 `ensureLogin()` 后 `listBindings()`; 按 `status` 渲染主账号 (静态"主账号"标签、无解绑) / 已绑定 ("解绑") / 未绑定 (P1 预留"绑定"灰显) Button, 成功后 `safeShowToast('绑定成功')` 并刷新; "解绑"走 `AlertDialog` 二次确认 (红色) `unbindAccount` + toast + 刷新; 无第三方绑定时展示空状态引导。
  4. `main\_pages.json` 增加 `pages/SettingsPage`、`pages/AccountBindingPage`。
- **\*\*验收点\*\*:**
  - "我的 → 设置 → 账号绑定"三级跳转正确, 返回链正常。
  - 列表首项恒为"鸿蒙账号 / 主账号", 无解绑按钮。
  - 绑定华为成功 toast + 列表即时刷新为"已绑定"; 取消授权状态不变。
  - 解绑走二次确认, 确定后刷新 + toast; 解绑失败 (如 409) 有明确提示。
  - 空状态 (无第三方绑定) 有引导而非白屏。

### ### T04 · 联调与端到端验收 【P0, 依赖 T01 + T03】

- **\*\*涉及文件\*\*:**
  - `backend/src/routes/account.test.ts` (新建: in-process 集成测试, mock prisma, 覆盖 09/403/404/正常路径, 复用 `auth.huawei.test.ts` 模式)
  - 前端真机/模拟器联调 (不落新文件, 按清单验证)

- **\*\*做什么\*\***:
  1. 编写 ``account.test.ts``: 缺 token→401; 同 unionID 跨账号→409; DELETE harmony→403; 不存在→404; 正常 bind/unbind 路径。
  2. 执行 ``prisma db push`` + ``prisma generate``, 启动后端, 跑测试。
  3. 前端连模拟器 (``hdc rport tcp:3000 tcp:3000``) 走完整链路: 绑定→刷新→解绑→刷新; 验证 toast 跳转。
- **\*\*验收点\*\***:
  - 后端测试全绿; 前端联调清单全通过 (含 409 占用提示、主账号不可解绑双保险、脱敏正确)。
  - 未登录态不展示"账号绑定"入口 (沿用 ``ensureLogin``)。

---

## ## 6. 依赖包列表

### ### 后端

- **\*\*无需新增任何 npm 包\*\***。``prisma`` / ``@prisma/client`` / ``express`` / ``jsonwebtoken`` / ``cors`` 均已存在; 华为换取 token 复用现有 ``services/huaweiAuth.ts`` (Node 原生 fetch, 零依赖)
- 仅需执行 ``npx prisma db push`` + ``npx prisma generate`` (属 prisma CLI 既有能力)。

### ### 前端

- **\*\*无需新增任何 import / SDK\*\***。``@kit.AccountKit`` (含 ``LoginWithHuaweiIDButton``、``loginComponentManager``) 已在 HarmonyOS SDK 内置; ``@kit.ArkUI``、``@kit.NetworkKit``、``@kit.BasicsServicesKit`` 均内置。
- 仅新增自有 ``.ets`` 文件 (见 §2.2)。

---

## ## 7. 共享知识 (跨文件约定)

- **\*\*provider 枚举值 (前后端单一来源)\*\***:
  - ``'harmony'``: 鸿蒙主账号 (展示、不可绑定/解绑)。
  - ``'huawei'``: 华为账号 (本期可绑定/解绑, `externalId = unionID`)。
  - ``'wechat'``: 微信 (P1, 本期不出现在后端逻辑, 仅类型预留)。
  - 后端 ``accountBindingService.ts`` 顶部定义 ``ALLOWED_BIND_PROVIDERS = ['huawei']`` (可列表, P0 仅 huawei; harmony 由列表合成); 前端 ``types.ets`` 定义 ``Provider`` 联合类型。
- **\*\*externalId 脱敏规则\*\*** (仅后端返回时脱敏, 存储为明文):
  - ``'harmony'``: ``openId`` 较短, 保留前 2 后 2, 中间 ``*****`` (长度 ≤6 则整体 ``*****``)。
  - ``'huawei'`` / ``'wechat'``: ``unionID/openid`` 较长, 保留末 4 位 → ``'*****' + raw.slice(-4)``。
  - 前端 **\*\*绝不\*\***接收明文 `externalId`; 列表展示直接用后端返回串。
- **\*\*错误码约定\*\*** (复用 ``utils/response.ts`` 的 ``CODE``):
  - ``401``: 缺/过期 token (auth 中间件统一返回)。
  - ``400``: 参数缺失 (缺 provider / `externalId` / 对 harmony 发起绑定)。
  - ``409``: 该 `externalId` 已被 **\*\*其他\*\*** 账号占用 (绑定冲突)。
  - ``403``: 试图解绑主账号 (harmony)。
  - ``404``: 待解绑的绑定不存在。
  - ``500``: 服务端异常。
- **\*\*token 获取方式\*\***: 前端从 ``AppStorage.get('authToken')`` 读取 (由 ``utils/auth.ets`` 的 ``session`` 持久化); ``api.ets`` 的 ``request`` 已自动注入 ``Authorization: Bearer <token>``, 业务动处理。
- **\*\*列表顺序约定\*\***: GET 返回数组固定顺序 = ``'harmony'`` (主账号) → ``'huawei'`` → ``'wechat'`` (P1) → ``playName`` 字典序兜底)。后端 ``listBindings`` 负责排序, 前端按序渲染即可。
- **\*\*`isPrimary` 语义\*\***: 仅合成的 harmony 主账号项 ``isPrimary=true``; ``UserBinding`` 表内行 `else`` (未来若支持"纯第三方注册"再启用)。
- **\*\*越权防护\*\***: 所有 ``UserBinding`` 读写必须以 ``req.userId`` (JWT 解析) 为 ``userId`` 过滤条件 (NOT { `userId`` } )。

---

## ## 8. 待明确事项 (需用户 / PM 确认)

1. **\*\*【关键技术风险】`HuaweiIDCredential` 是否在本期真机直接暴露 `unionID`? \*\***

----- 第 22 页 -----

现有 `HuaweiLoginButton` 回调仅取 `authorizationCode`，未使用 `unionID`；华为 Account 在 `LoginWithHuaweiIDButton` 场景下，`unionID` 是否随凭证回调下发需\*\*真机联调确认\*\*。

- 若\*\*直接暴露\*\*：本设计（前端取 `unionID` → POST `{provider, externalId}`）成立。
- 若\*\*不暴露\*\*（仅给 code）：需改为「前端取 `code` → 后端换 `unionID`」的绑定变体，POST {code, provider, code}，后端复用 `huaweiAuth.ts` 的 `exchangeCodeForToken + fetchHuaweiUserProfile`，取 `unionID` 后再写表。届时同步调整接口契约与 `HuaweiBindButton` 暴露的回调（改为 `onHuaweiBind`）。\*\*建议 T02/T03 开工前先在真机验证一次\*\*，以锁定契约形态。

2. \*\*微信 (P1) 授权形态\*\*：原生 SDK vs 应用内 Web 弹层，本期不做，P1 启动时复评（不影响 P0 数据）。

3. \*\*绑定成功是否触发站内安全消息 (P1-2)\*\*：本期 P0 仅 `safeShowToast` 提示，不发站内信；是否待 PM 定。

4. \*\*主账号 `boundAt` 取值\*\*：本期用 `User.createdAt` 作为主账号"创建（本机登录）"时间，是否绑定时间"需另存字段（本期不引入）。

5. \*\*软删除 / 审计\*\*：本期 `UserBinding` 硬删；P2 异常检测 (P2-1) 如需"失效但保留记录"，再引入 `status` 字段。

6. \*\*`User.unionID` 唯一约束与绑定同步的一致性\*\*：本期保证"同一 unionID 仅属于一个账号"（冲突一个华为号关联多个大蓝书号"（多主），需放开 `unionID` 唯一约束——本期按"一对一"落地。

---

> 文档结束。配套图：`entry/Docs/class-diagram.mermaid`（类/结构关系）、`entry/Docs/sequence-diagram.mermaid`（绑定/解绑时序）。

// ===== ./entry/Docs/design-content-moderation.md =====  
# 系统设计：内容审核 & 举报系统

> 项目：大蓝书 HarmonyOS NEXT 应用（男性生活经验社区）  
> 版本：v1.0  
> 日期：2025-07  
> 关联文档：[PRD](./prd-content-moderation.md)  
> 架构师：高见远

---

## ## 1. 实现方案总览

### ### 1.1 整体架构

本系统在现有「大蓝书」架构 (Node.js + Express + TS + Prisma + MySQL 后端 / HarmonyOS NEXT V1 前端) 基础上，新增三道内容安全防线：

1. \*\*敏感词前置过滤\*\*：发帖/评论前由后端 `SensitiveWordService` 单例检测，命中即返回 400，不予发布。
2. \*\*用户举报\*\*：用户在帖子详情页/评论区通过「…」→ ActionSheet → 举报弹窗提交举报；举报累计达 3 次（含）进入待审核队列。
3. \*\*开发者审核\*\*：开发者通过 admin API (Postman/curl) 查看待审帖子并 approve/reject，系统同步更新状态。

数据流：`前端提交` → 敏感词检测 → 落库 status=1 → 用户举报 → reportCount++ → 达阈值 status=2 → 进入审核 → status=1(恢复)/status=2(拒绝) → 系统通知

### ### 1.2 关键技术选型

决策点	选型	理由
敏感词检测	**自实现 Trie 树** (非 Set、非外部包)	ToolGood.Words 是 C# 原生无 Node.js 支持，在 5000+ 词时 O(nxm) 性能差；Trie 实现 O(nxL) (n=文本长度, L=最长词长度)，代码量约 600 行，速率均优于 Set
敏感词库	**纯文本文件** `backend/data/sensitive-words.txt` + `backend/data/gender-words.txt`	每行一个词，启动时 fs.readFileSync 加载到 Trie，无热更新需求 (P1 再做)
admin 鉴权	**环境变量 `ADMIN_USER_IDS`** + `adminAuth` 中间件	MVP 单人审核，校验 `ADMIN_USER_IDS` 即可

Id` 是否在列表中, 403 拒绝。复用现有 `auth` 中间件先解析 token |  
| 举报阈值 | \*\*环境变量 `REPORT\_THRESHOLD`\*\* (默认 3) | `reportService.createReport` 中  
element reportCount 后判断是否 >= 阈值 |  
| 举报幂等 | \*\*Prisma `@@unique([reporterId, targetType, targetId])`\*\* | 数据库级保证同  
同一内容仅举报一次; 冲突时捕获 Prisma error 返回 409 |  
| 前端 ActionSheet | `this.getUIContext().showActionMenu()` | API 24 推荐方式; 若不可用  
CustomDialog (见风险评估) |  
| 前端举报弹窗 | `@CustomDialog` 装饰器 + `CustomDialogController` | ArkTS V1 原生弹窗机  
义布局 |

### ### 1.3 敏感词库加载策略

----- 第 23 页 -----

...

后端进程启动

```
→ SensitiveWordService 单例初始化
→ fs.readFileSync('backend/data/sensitive-words.txt') // 通用敏感词
→ fs.readFileSync('backend/data/gender-war-words.txt') // 男女对立引战词
→ 合并去重 → 逐词插入 Trie 树
→ 全局单例就绪, 后续请求直接调用 checkText()
...
```

- 文件路径基于 `process.cwd()` (即 `backend/` 目录)
- 若文件不存在, 日志告警但不崩溃 (降级为空词库, 不拦截)
- 词库预计 5000-20000 词, Trie 内存占用 < 2MB

---

## ## 2. 文件列表及改动类型

### ### 后端 - 新增文件

文件	类型	说明
backend/src/services/sensitiveWordService.ts	新增	敏感词检测单例服务 (Trie 树 + Text)
backend/src/services/reportService.ts	新增	举报 CRUD + 阈值触发自动下架
backend/src/services/moderationService.ts	新增	待审帖子列表 + 审核操作 (approve)
backend/src/middleware/adminAuth.ts	新增	admin 鉴权中间件
backend/src/routes/admin.ts	新增	admin 审核 API 路由
backend/data/sensitive-words.txt	新增	通用敏感词库 (种子文件, 约 50 词)
backend/data/gender-war-words.txt	新增	男女对立引战词库 (种子文件, 约 20 词)
backend/src/services/sensitiveWordService.test.ts	新增	敏感词服务单元测试
backend/src/services/reportService.test.ts	新增	举报服务单元测试
backend/src/routes/admin.test.ts	新增	admin 路由集成测试

### ### 后端 - 修改文件

文件	类型	说明
backend/prisma/schema.prisma	修改	新增 Report 模型 + Post.reportCount + Comment.status/reportCount
backend/src/config/env.ts	修改	新增 adminUserIds、reportThreshold
backend/src/services/postService.ts	修改	createPost 接入敏感词前置检测
backend/src/services/commentService.ts	修改	createComment 接入敏感词前置检测; 1
backend/src/services/notificationService.ts	修改	新增 notifySystem 函数
backend/src/routes/posts.ts	修改	新增 POST /:id/report 路由
backend/src/routes/comments.ts	修改	新增 POST /comments/:id/report 路由
backend/src/app.ts	修改	挂载 admin 路由



| `backend/.env.example` | 修改 | 新增 ADMIN\_USER\_IDS、REPORT\_THRESHOLD |

### 前端 - 新增文件

| 文件 | 类型 | 说明 |

----- 第 24 页 -----

|-----|-----|-----|

| `entry/src/main/ets/components/ReportDialog.ets` | 新增 | 举报弹窗组件 (理由单选 + 补

交/取消) |

### 前端 - 修改文件

| 文件 | 类型 | 说明 |

|-----|-----|-----|

| `entry/src/main/ets/models/types.ets` | 修改 | 新增 ReportReason 类型 + ReportBody

□ |

| `entry/src/main/ets/services/api.ets` | 修改 | 新增 reportPost/reportComment; 修改

quest 错误处理以透传后端 message |

| `entry/src/main/ets/pages/DetailPage.ets` | 修改 | 顶部「…」按钮 + ActionSheet + 举

+ 待审核横幅 + 敏感词 Toast |

| `entry/src/main/ets/components/CommentList.ets` | 修改 | 每条评论右侧「…」按钮 + Act

et + 举报回调 |

| `entry/src/main/ets/pages/MessagePage.ets` | 修改 | system 类型通知渲染样式优化 (系统图

区分) |

\*\*合计: 新增 11 个文件, 修改 14 个文件.\*\*

---

## 3. 数据模型设计

### 3.1 新增 Report 模型 (Prisma schema)

```
``prisma
// 举报记录表
model Report {
 id Int @id @default(autoincrement())
 reporterId Int // 举报人 userId
 targetType String @db.VarChar(10) // 'post' | 'comment'
 targetId Int // 目标帖子或评论 ID
 reason String @db.VarChar(20) // 举报理由枚举值 (见 §9.2)
 description String? @db.VarChar(200) // 补充说明 (reason='other' 时必填)
 status String @default('pending') @db.VarChar(20) // pending | resolve
 dismissed Boolean @default(false)
 createdAt DateTime @default(now())
 resolvedAt DateTime?

 @@unique([reporterId, targetType, targetId]) // 同一用户对同一内容仅能举报一次 (幂等)
 @@index([targetType, targetId]) // 按内容查举报记录
 @@index([status, createdAt]) // 审核队列按状态+时间排序
}
```

### 3.2 Post 模型新增字段

```
``prisma
// 在现有 Post 模型中新增:
reportCount Int @default(0) // 被举报累计次数 (达阈值自动 status=0)
```

> Post 现有 `status Int @default(0)` 字段无需改动 (0-待审核 1-已发布 2-已拒绝), 语义完全复

### ### 3.3 Comment 模型新增字段

----- 第 25 页 -----

```
```prisma
// 在现有 Comment 模型中新增:
status      Int    @default(1)  // 1-正常 0-隐藏(待审核) 2-已删除
reportCount  Int    @default(0)  // 被举报累计次数
```
```

> \*\*迁移注意\*\*: Comment 表已有数据新增 `status` 字段, `@default(1)` 使存量评论自动标记为「正

### ### 3.4 关系图 (Mermaid classDiagram)

```
```mermaid
classDiagram
    class Report {
        +Int id
        +Int reporterId
        +String targetType
        +Int targetId
        +String reason
        +String~?~ description
        +String status
        +DateTime createdAt
        +DateTime~?~ resolvedAt
    }

    class Post {
        +Int id
        +Int userId
        +String title
        +String~?~ content
        +Int status
        +Int reportCount
        +DateTime createdAt
    }

    class Comment {
        +Int id
        +Int postId
        +Int userId
        +String content
        +Int status
        +Int reportCount
        +DateTime createdAt
    }

    class SensitiveWordService {
        -TrieNode root
        -boolean loaded
        +loadWords(words: string[]) void
        +checkText(text: string) boolean
    }

```

----- 第 26 页 -----

```
class ReportService {
    +createReport(reporterId, targetType, targetId, reason, description?) Re
port
    +listReportsByTarget(targetType, targetId) Report[]
}
```

```

    +resolveReportsByTarget(targetType, targetId, newStatus) void
}

class ModerationService {
    +listPendingPosts(page, limit) ListResult
    +moderatePost(postId, action, reason?) void
}

class NotificationService {
    +createNotification(input) void
    +notifySystem(userId, content, postId?) void
    +notifyOnComment(postId, actorId) void
}

Report ..> Post : targetType='post' targetId=post.id
Report ..> Comment : targetType='comment' targetId=comment.id
ReportService --> Report : CRUD
ModerationService --> ReportService : resolveReportsByTarget
ModerationService --> NotificationService : notifySystem
ReportService --> NotificationService : 阈值触发时 notifySystem
SensitiveWordService ..> Post : createPost 前置检测
SensitiveWordService ..> Comment : createComment 前置检测
...

```

3.5 建表迁移命令

```

```bash
cd backend
npx prisma db push # 禁止 prisma migrate dev (bigbluebook 用户无 CREATE DATABASE 本
)
npx prisma generate # 重新生成 client
⚠️ 改 schema 后必须重启后端进程 (tsx watch 会自动重启) , 让新 PrismaClient 生效
```

```

4. 后端服务设计

4.1 sensitiveWordService.ts (新增 - 单例)

```

```typescript
// backend/src/services/sensitiveWordService.ts

// Trie 节点
interface TrieNode {
 children: Map<string, TrieNode>;
 ----- 第 27 页 -----
 isEnd: boolean;
}

class SensitiveWordService {
 private root: TrieNode;
 private loaded: boolean;

 constructor() {
 this.root = { children: new Map(), isEnd: false };
 this.loaded = false;
 }

 // 启动时调用: 从文件加载词库并构建 Trie
 // filePaths: 词库文件路径数组 (相对于 process.cwd())

```

```

loadFromFiles(filePaths: string[]): void {
 // 逐文件 readFileSync → split('\n') → trim → 过滤空行 → insert
 // 多文件合并去重
 // this.loaded = true
}

// 向 Trie 插入单个词 (转小写)
private insert(word: string): void { ... }

// 检测文本是否包含任意敏感词
// 返回 true=命中 false=未命中 (不暴露具体命中词)
checkText(text: string): boolean {
 // 1. text.toLowerCase()
 // 2. 遍历每个起始位置 i, 沿 Trie 向下匹配
 // 3. 任一节点 isEnd=true → return true
 // 4. 全部遍历完无命中 → return false
}

isLoading(): boolean { return this.loaded; }
}

// 全局单例
export const sensitiveWordService = new SensitiveWordService();

调用契约:
- `checkText(text: string): boolean` - 发帖检测 `title + ' ' + content`; 评论检测 `content`
- 启动时在 `app.ts` 或 `index.ts` 中调用 `sensitiveWordService.loadFromFiles(['data/sensitive-words.txt', 'data/gender-war-words.txt'])`

4.2 reportService.ts (新增)

```typescript
// backend/src/services/reportService.ts
import { prisma } from '../prisma';
import { sensitiveWordService } from '../sensitiveWordService';
import { notifySystem } from '../notificationService';

----- 第 28 页 -----
import { env } from '../config/env';

export type TargetType = 'post' | 'comment';
export type ReportReason = 'political' | 'pornographic' | 'personal_attack' |
  'gender_war' | 'advertisement' | 'spam' | 'other';

// 创建举报 (幂等: 同一用户同一内容仅一次)
// 抛出错误: { reason: 'conflict' } 重复举报; { reason: 'not_found' } 目标不存在
export async function createReport(params: {
  reporterId: number;
  targetType: TargetType;
  targetId: number;
  reason: ReportReason;
  description?: string;
}): Promise<{ report: any; autoTakenDown: boolean }>;
// 逻辑:
// 1. 校验 reason='other' 时 description 必填
// 2. 校验目标是否存在 (Post/Comment findUnique)
// 3. prisma.report.create({ ... }) - unique 约束冲突 → 抛 conflict
// 4. 递增目标 reportCount (Post.update 或 Comment.update)
// 5. 判断 reportCount >= env.reportThreshold → 更新 status=0 + notifySystem 作者
// 6. 返回 { report, autoTakenDown }

```

```
// 按目标查询举报记录 (供审核使用)
export async function listReportsByTarget(
  targetType: TargetType,
  targetId: number
): Promise<any[]>;

// 批量更新某目标的所有 pending 举报状态 (审核时调用)
export async function resolveReportsByTarget(
  targetType: TargetType,
  targetId: number,
  newStatus: 'resolved' | 'dismissed'
): Promise<void>;
// 逻辑: prisma.report.updateMany({
//   where: { targetType, targetId, status: 'pending' },
//   data: { status: newStatus, resolvedAt: new Date() }
// })

// 获取某目标的所有举报人 ID (用于审核后通知举报人)
export async function getReporterIdsByTarget(
  targetType: TargetType,
  targetId: number
): Promise<number[]>;
```

4.3 moderationService.ts (新增)

```
```typescript
```

```
----- 第 29 页 -----
```

```
// backend/src/services/moderationService.ts
import { prisma } from '../prisma';
import { resolveReportsByTarget, getReporterIdsByTarget } from '../reportService';
import { notifySystem } from '../notificationService';

// 待审核帖子列表 (status=0)
export async function listPendingPosts(
 page: number = 1,
 limit: number = 20
): Promise<{ list: any[]; pagination: { page: number; limit: number; total: number } }>;
// 逻辑: prisma.post.findMany({
// where: { status: 0 },
// orderBy: { createdAt: 'desc' },
// skip, take,
// include: { user: { select: { id, nickname, avatar } } }
// })

// 审核帖子
// action: 'approve' → status=1, reports → dismissed
// action: 'reject' → status=2, reports → resolved
// 抛出错误: { reason: 'not_found' } 帖子不存在; { reason: 'invalid_status' } 非待审状态
export async function moderatePost(
 postId: number,
 action: 'approve' | 'reject',
 reason?: string
): Promise<void>;
// 逻辑:
// 1. 查帖子, 校验 status===0
// 2. Post.update({ status: action==='approve' ? 1 : 2 })
// 3. resolveReportsByTarget('post', postId, action==='approve' ? 'dismissed' :
```

```
'resolved')
// 4. 通知作者:
// approve → notifySystem(post.userId, `你的帖子 《${post.title}》 已通过审核`, postId)
// reject → notifySystem(post.userId, `你的帖子 《${post.title}》 未通过审核${reason}`)
原因: ' + reason : '}', postId)
// 5. 通知所有举报人:
// reporterIds = await getReporterIdsByTarget('post', postId)
// for (id of reporterIds) notifySystem(id, '你的举报已处理', postId)
// (不透露审核结果, 统一告知"已处理")
`
```

### 4.4 notificationService.ts (修改 - 新增 notifySystem)

```
`typescript
// 在现有 notificationService.ts 中新增:

// 系统通知 (actorId=null, type='system')
export async function notifySystem(
 userId: number,
 content: string,
 postId?: number | null
): Promise<void>;
----- 第 1 页 -----
立项日期: 2026年7月9日
开发者: 个人开发者 (个体工商户主体)
目标平台: HarmonyOS NEXT (鸿蒙原生应用)
产品定位: 男性生活经验与处事方法论的分享社区
```

## 一、产品概述

### 1.1 产品使命

**「大蓝书」是一个专注于男性理性经验分享的平台。**

它的核心价值是：帮助男性用户在做消费决策、解决生活问题、提升个人能力时，能够找到真实、硬核、有方法论

### 1.2 与小红书的核心差异

维度	小红书	大蓝书
内容基调	美好生活、种草、情绪共鸣	硬干货、避坑、方法论、实操
内容形式	图文/视频混排，氛围感优先	结构化内容（优点/缺点/步骤）
用户动机	“逛一逛”、“被种草”	“搜一下”、“查攻略”、“做对比”
互动方式	点赞、收藏、评论区夸夸	顶、抄作业、事实补充、理性辩论
社区氛围	精致、共情、温和	硬核、求真、有图有真相

### 1.3 目标用户画像

- **年龄**: 18~35岁
- **性别**: 男性 (占比目标 > 85%)
- **特征**:
  - 有明确的消费决策需求 (买数码、买车、买家电)
  - 喜欢自己动手解决问题 (修东西、做饭、健身)
  - 关注个人成长 (职场、理财、学习)
  - 反感“情绪化输出”，偏好“数据说话”
- **典型场景**:
  - “想买一款蓝牙耳机，先上大蓝书搜测评”
  - “家里水龙头漏水，上大蓝书看教程”
  - “刚入职一家新公司，上大蓝书学职场沟通”

## 二、核心功能模块

### 2.1 功能总览 (MVP版本)

模块	核心功能	优先级
***首页**	推荐流（基于兴趣标签）、最新流、热榜	P0
***标签导航**	30个话题标签横向滚动，点击筛选	P0
***内容详情页**	图文展示、结构化字段展示、评论区	P0
***发布器**	选体裁→选标签→填结构化表单→发布	P0
***消息通知**	评论/顶/抄作业的互动提醒	P1

----- 第 1 页 -----

***个人主页**	头像/昵称/发布的帖子/收藏的帖子	P1
***搜索**	关键词搜索帖子标题和正文	P1

### 2.2 首页设计

\*\*\*布局结构（从上到下）：\*\*\*

...

状态栏（透明毛玻璃）	
[大蓝书 Logo]	[消息]